Evidence-Augmented Answer Selection

DASE7506 - MP2 | Selecting Reliable LLM Reasoning

Zhu Jinting | Student ID: 3036709066 | Fall 2026

84.3683%
Official test accuracy

394 / 467
Correct selections

23.62 KiB
Inference assets

Abstract

This project selects one of four supplied solutions to a mathematics question. The final method combines a linear scorer over fixed solution embeddings with a second scorer over explicit mathematical evidence. A small fusion model is trained using out-of-fold scores. All learned parameters and preprocessing statistics are fitted on the supplied training split. The frozen fourth version obtains 394/467 correct selections (84.36830835117773%) on the official test bank. It improves training out-of-fold hard-question accuracy from 81.03% to 82.59% and validation accuracy from 93.36% to 94.24% relative to the third-version baseline. These are course-bank selection results, not evidence of full reasoning verification.

1. Task, data and research question

The inputs are a question, four candidate texts, and four 1,024-dimensional float32 embeddings. The output is one Python integer in 0..3, preserving bank order. The official scorer judges only the final numeric answer, including equivalent fractions and decimals. A question can have several correct candidates. At least one correct answer is supplied for each group.

Split	Questions	Purpose
Training	4,755	Fitting and cross-validation
Validation	798	Development evaluation
Test	467	Official scoring after each version freeze

The course protocol is dase7506-mp2-open-hard-verifier-v5. Every test question has at least two distinct valid parsed answers; training and validation also contain easier groups. In this report, hard means at least two distinct valid parsed answers, excluding parsing failures. Minority means that the correct-answer group is smaller than the largest answer group. There are 2,114 hard and 367 minority questions in training, and 355 hard and 60 minority questions in validation.

Research hypothesis

Explicit evidence about arithmetic and answer consistency can complement fixed embeddings when candidate solutions disagree. The hypothesis is tested using individual feature-block comparisons, an evidence-only model, and a cross-fitted fusion model. Features are evaluated empirically rather than treated as proofs of correctness.

2. Method and mathematical evidence

Frozen method: evidence_method.py

The base scorer uses the 1,024 supplied embedding coordinates plus 13 existing text and consensus features. These include candidate length, numeric and formatting counts, selected wording cues, parsed-answer validity and answer frequency. The evidence scorer uses 46 additional features derived only from the question and candidate text.

Block	Count	Information represented
Arithmetic	15	Checked equations, exact/rounded agreement, mismatch rates and residuals, operator counts
Answer consistency	12	Final answer versus checked calculations; conflicting final markers; numeric properties
Question constraints	19	Discrete counts, remaining/average/total cues, percentages, time conversion, units and number overlap

Arithmetic is evaluated through a bounded Python AST whitelist with Fraction arithmetic; candidate code is never executed. Supported expressions include addition, subtraction, multiplication, division, parentheses, decimals and simple LaTeX fractions. Unsupported expressions are unknown, not automatically incorrect. Exact equality and decimal rounding are recorded separately. Markdown bullets, bold formatting, boxed answers and written percentage-point conventions are handled explicitly.

Question constraints are soft English-language heuristics, not a semantic solver. For example, a noninteger answer to a discrete-count question or a missing subtraction cue may be informative but is not conclusive. Expressions involving units or ambiguous boundaries may still be misinterpreted. Only failure of the unchanged official numeric parser is used as a hard exclusion.

Answer-group objective

For candidate j, let s_j be its linear score and n_j the number of candidates sharing its valid parsed answer. Define z_j = s_j - log(n_j). Invalid candidates receive zero probability. The per-question loss is:

```
L = -log[ sum_{j correct} exp(z_j) / sum_{j valid} exp(z_j) ]
    + (lambda / 2) * ||w||^2
```

All correct candidates contribute to the numerator. If every valid candidate is correct, the selection loss is zero, so that question can be omitted from loss optimization. At inference, each answer group receives log(mean(exp(s_j))) over its members. The highest-scoring candidate inside the winning group supplies the returned index.

Fusion

The fusion scorer has four inputs: the centered base score, the centered evidence score, valid-answer frequency, and the interaction between the centered base score and whether a checked equation exists. Centering is within each question over valid candidates. Separate means, scales and weights are saved for the base, evidence and fusion scorers.

3. Experimental design and ablations

Selection on training out-of-fold predictions

Version 4 uses three outer folds of complete questions, stratified by correctness count, answer diversity and minority status. All four candidates stay together. The folds match version 3. All standardization statistics are fitted on the corresponding training partition. For fusion, three inner folds generate predictions for training the meta scorer; outer evaluation labels enter neither base-model nor fusion fitting.

The final fusion model is fitted on all training questions: base and evidence scorers are refitted, and the meta scorer is trained on outer out-of-fold predictions. Fixed optimization settings are L2 coefficient 0.03, Adam learning rate 0.003, batch size 128, 80 epochs and averaging of the last 10 epochs. Adam uses beta1=0.9, beta2=0.999 and epsilon=1e-8. The seed is 20261027. No validation-based early stopping is used.

Before the run, protocol.json specified seven variants. An alternative had to improve pooled hard-question accuracy and improve at least two outer folds over the baseline. Qualifying variants were ranked by hard accuracy and then overall accuracy; otherwise the baseline would be retained. Configuration and asset hashes were saved before loading the validation split for this round.

Method	Overall (%)	Hard (%)	Minority (%)
Embedding + text baseline	91.57	81.03	53.13
+ Arithmetic evidence	91.99	81.98	57.22
+ Answer consistency	91.88	81.74	55.04
+ Question constraints	91.48	80.84	53.13
+ All 46 features	92.20	82.45	57.49
Evidence only	87.87	72.71	68.66
Nested cross-fitted fusion	92.26	82.59	52.32

The selected fusion improves all three outer hard-question folds: 79.63 -> 82.74%, 80.54 -> 81.96%, and 82.93 -> 83.07%. Across all training out-of-fold predictions it fixes 89 baseline errors and introduces 56, a net gain of 33. Hard selections increase from 1,713/2,114 to 1,746/2,114.

What the comparisons show

Arithmetic and answer-consistency additions each improve hard accuracy. Question constraints alone slightly reduce it. All evidence together is useful, while evidence alone is substantially weaker overall. Fusion has the highest pooled hard accuracy, but its minority accuracy is slightly below the base model. Therefore the result does not support an improvement on every difficult subgroup.

These are feature-addition comparisons under fixed settings, plus an evidence-only ablation. They do not isolate every possible interaction inside fusion. The out-of-fold results participate in selection and should be treated as development estimates rather than an unbiased final assessment.

4. Validation, official test and error analysis

Final version frozen before its test run

Evaluation	Version 3	Version 4
Validation overall	745/798 (93.36%)	752/798 (94.24%)
Validation hard	302/355 (85.07%)	309/355 (87.04%)
Validation minority	33/60 (55.00%)	37/60 (61.67%)

On validation, version 4 fixes 10 version-3 errors and introduces three. The net gain is seven questions. This development result is distinct from the official test score below; validation accuracy is not the submission metric.

Frozen version	Official test correct	Accuracy (%)
v1: linear baseline	384/467	82.22698073
v2: weighted ridge	382/467	81.79871520
v3: answer-group model	381/467	81.58458244
v4: evidence fusion	394/467	84.36830835

The exact official accuracy for the packaged version is 84.36830835117773. The prediction process and independent scorer were run after verifying the pre-test frozen hashes. Version 4 selects ten more correct answers than version 1 and thirteen more than version 3. It still misses 73 test questions. These aggregate differences are descriptive; no claim of statistical significance or generalization to an unseen audit bank is made.

Why arithmetic evidence must remain soft

Of 19,020 training candidates, 17,508 (92.05%) contain at least one checked equation. Numerical inconsistencies are flagged in 619 candidates, but 188 of those still have the correct final answer under the official scorer. Rejecting every trace with an intermediate error would therefore discard some valid selections.

Training trace example	Observation
24 - 5 + 8 = 25	The stated result is 25; the arithmetic result is 27.
2000 + 750 + 250 = 2900	The sum should be 3000. This is a potentially useful mismatch.
88 - (1/4)*88 = 352/4 - 352/4 = 88 - 22 = 66	The intermediate equality is inconsistent, although the endpoint is 66.

A candidate may calculate correctly but omit a condition, use the wrong base quantity, or solve a different interpretation. Conversely, its last visible equation may be an intermediate result rather than the requested answer. The learned fusion balances these imperfect signals with the embedding scorer. It does not establish that the reasoning chain is valid.

5. Resources, implementation and reproduction

CPU-only inference with all assets included

Item	Recorded result
Hardware	Intel Core Ultra 9 290HX Plus; Windows 11
Software	Python 3.12.14; NumPy 2.3.5
Final inference assets	24,191 bytes (23.62 KiB), uncompressed
Threads	4 via OMP, OpenBLAS and MKL environment settings
v4 training and cross-validation	39.66 s; final feature extraction 12.75 s
Earlier evidence-parser extraction passes	Approximately 34 s combined
Full test prediction + independent scoring	2.809 s, including fresh process startup

The course reference Xeon Platinum 8457C has not been tested locally; the local measurement does not certify reference-machine eligibility. The 24,191 bytes include three NPZ files and config.json. Training caches, fold assignments and out-of-fold predictions are analysis artifacts and are not read during inference. No external weights or datasets are used.

Earlier recorded training/search times were 5.07 s for v1, 17.96 s for v2 and 53.56 s for v3. These timings exclude interactive code development. Versions 1 and 2 each compared 60 configurations plus their stated baselines; version 3 compared eight configurations. Final-stage comparisons and all official score records are included in the package.

Reproduce the frozen score without training

Use Python 3.12. From code/, install the pinned requirements, set the three thread environment variables to 4, and run the following. The supplied package includes all course data and inference assets.

```
python -m pip install -r requirements.txt
python check_data.py
python evaluate_evidence.py
```

evaluate_evidence.py verifies frozen hashes, launches predict.py with evidence_method.py and runs the independent score_predictions.py. Expected result: correct=394, n=467, accuracy=84.36830835117773. Exact PowerShell and POSIX commands are in the root README.

Reproduce training

```
python check_evidence.py
python audit_evidence.py
python train_evidence.py
python verify_evidence.py
```

Training should be run in a separate extracted copy because it overwrites generated run files. The original course requirements pin NumPy 2.5.3; this submission pins the actually tested NumPy 2.3.5 and retains the original requirement file in provenance. Reproduction under the course pin and reference CPU remains unverified.

Checks cover 625 arithmetic cases, frozen hashes, candidate permutations, new IDs, batch independence and asset-only inference. A clean packaging-time training run reproduced all four asset files byte-for-byte and all 467 predictions. The official loader, parser and scorer remain unchanged.

6. Development history, limitations and disclosure

Transparent record of all evaluated versions

Development and test exposure

Version 1 selected a ridge configuration on validation and was tested. Development then continued after its aggregate test score was known. Version 2 explored mixed-question training, minority weighting and pairwise ranking, was selected on validation, and was tested. Version 3 introduced question-level cross-validation and an answer-group objective, was frozen and tested. Version 4 added mathematical evidence and nested fusion; its selection used training out-of-fold criteria, followed by validation and a frozen official test. Version 4 is the packaged final method.

Test references were not fitted as training labels, used to construct answer lookups, or inspected as per-question error targets for later models. However, aggregate test results from earlier versions were visible while the overall project continued. The development history therefore does not constitute a pristine one-shot held-out test protocol. All scores are retained rather than presenting only the best run.

Limitations

Rule coverage is partial. Variables, powers, complex verbal arithmetic, and some unit conventions are unsupported or ambiguous. Incorrect extraction remains possible. The heuristic constraint block does not understand the full problem. Correct arithmetic cannot prove correct problem interpretation, and a correct final answer does not prove correct reasoning.

The validation bank has been reused across development rounds. Cross-validation was used to select later configurations, so its selected scores also contain selection effects. Minority cases are relatively few, and the final method did not improve their training out-of-fold accuracy. No statistical significance, calibration or causal reasoning claim is made. Results apply to the fixed course candidate-selection bank.

AI assistance and reuse

OpenAI Codex provided substantive assistance with implementation, feature design, experiment execution, code review, analysis, English report drafting and packaging. This assistance is disclosed explicitly. The official course data, common.py, predict.py, score_predictions.py and check_data.py are reused from the supplied assignment. The student is responsible for understanding and verifying the submission.

Sources and attribution

[1] DASE7506 MP2 assignment guide and code instructions supplied with the coursework. Copies are retained in provenance/. Protocol: dase7506-mp2-open-hard-verifier-v5.

[2] GSM8K questions and references: OpenAI, 2021. Source: https://github.com/openai/grade-school-math. The supplied MIT LICENSE.txt and NOTICE.txt are retained in code/data/.

[3] Candidate traces and fixed embeddings were prepared by the course using Qwen/Qwen3-0.6B, revision c1899de289a04d12100db370d81485cdf75e47ca, as recorded in the supplied manifest. No model weights are distributed or used by this submission.

[4] Experiment evidence: code/runs/evidence_v4/ contains protocols, fold-level metrics, ablations, paired comparisons, hashes, validation/test JSON and resource records. Earlier version records are under records/history/. The package manifest identifies the exact delivered files.